AgentRearrange: A General-Purpose Multi-Agent Orchestrator

Kye Gomez[1,*], Kuanting Kuo[1], Aditya Chaudhary[1], Hugh Nguyen[1], Swarms Research Team[1]

[1]Swarms [*]Project Lead

Correspondence: kye@swarms.world

Abstract

This report introduces **AgentRearrange**, a multi-agent orchestration primitive in the Swarms framework that lets engineers express arbitrary mixtures of sequential and concurrent agent execution through a tiny, readable flow grammar. Where dedicated structures such as `SequentialWorkflow` and `ConcurrentWorkflow` fix a single topology at construction time, AgentRearrange separates *topology* from *instantiation*: a workflow is a string like `"researcher -> writer, reviewer -> editor"`, parsed at runtime into an executable schedule. We describe the motivation behind the abstraction, the parsing and scheduling model, the team-awareness mechanism that lets each agent see its upstream and downstream neighbors, and the supporting features for batched, concurrent, asynchronous, and token-streaming execution. We then present architectural diagrams for the canonical patterns AgentRearrange enables (linear pipelines, fan-out/fan-in, diamonds, and human-in-the-loop hops), followed by runnable code examples. The implementation is open-source under Apache-2.0, ships in the `swarms` package on PyPI, and is available at `github.com/kyegomez/swarms`.

1 Introduction

The dominant pattern in multi-agent systems literature is to pick a topology (sequential, concurrent, hierarchical, debate) and bake it into a class. Each topology becomes its own primitive: `SequentialWorkflow` for pipelines, `ConcurrentWorkflow` for fan-outs, `HierarchicalSwarm` for boss-worker delegation, and so on. This design is clean for textbook examples but becomes awkward the moment a real-world workflow needs a *mix* of behaviors, for instance a research agent feeding two parallel writers whose drafts are then merged by an editor.

The Swarms framework introduces **AgentRearrange**, a structure whose only configuration is a flow string and a list of agents. The flow string is a compact domain-specific language (DSL) with two operators: `->` denotes sequential dependence ("the right side waits for the left"), and `,` denotes concurrent execution on the same upstream input. These two operators compose, so workflows like `"a -> b, c -> d"` ("a runs, then b and c run in parallel, then d aggregates") read almost like ASCII art of the underlying DAG.

The contribution of AgentRearrange is not novel scheduling theory; topological execution of a DAG is decades old. The contribution is *ergonomic*: the abstraction lowers the cognitive cost of

describing a workflow to the point where engineers can iterate on topology as quickly as they iterate on prompts. A workflow is no longer a class hierarchy; it is a string. Reordering agents, inserting a parallel review step, or fanning out to multiple model providers is a one-line edit. This matters in practice because most multi-agent systems do not arrive at their final shape on day one; they evolve through experimentation, and the friction of restructuring an orchestration is often the bottleneck.

This report walks through why the abstraction exists, how it is implemented in `swarms/structs/agent_rearrange.py`,[1] the patterns it enables, and the supporting features (streaming, batched execution, asynchronous integration) that turn the core primitive into a production-ready building block.

2 Background and Motivation

Early Swarms users repeatedly hit the same wall. They would prototype a research workflow with `SequentialWorkflow`, get it working, then realize the middle stage should actually be three agents running in parallel and merging their outputs. The fix required swapping orchestration classes, refactoring the agent list, and re-threading the conversation history. The workflow shape was *baked into the orchestrator type*, so every shape change cost a rewrite.

A second class of users wanted to A/B test workflow topologies. Given five agents, does `a -> b -> c -> d -> e` produce better outputs than `a -> (b, c) -> (d, e)`? Or than `a -> b -> (c, d) -> e`? Answering these questions required maintaining several orchestrator instances, each in a slightly different shape, often with subtle drift in how they handled conversation state.

The third pain point was *readability*. Reading a 30-line orchestrator setup with conditional graph construction tells you very little about what the workflow actually does. By contrast, the string `"researcher -> analyst, critic -> editor -> publisher"` tells a reader exactly how data flows in fewer characters than a tweet.

AgentRearrange was designed to collapse these three problems into one. The design constraints were:

- **Topology as data, not code.** Flow lives in a string that can be edited, generated, or fed from a config file, not a class hierarchy.
- **Grammar minimal enough to memorize.** Two operators, both ASCII, both readable left-to-right.
- **One conversation, one source of truth.** Every agent reads from and writes to the same `Conversation` object so context propagates correctly across both sequential and parallel hops.
- **No special-casing.** Every other Swarms structure ultimately reduces to a flow string; if AgentRearrange covers the general case, the dedicated structures become specialized factories on top of it.
- **Production-ready surface.** Sync, async, streaming, batched, and concurrent execution all available without changing the flow.

The result is the primitive described in the remainder of this report.

[1] `swarms/structs/agent_rearrange.py` on GitHub.

3 Core Design: The Flow DSL

The flow string is the heart of AgentRearrange. Its grammar fits on one line:

```
flow      := step ("->" step)*
step      := agent_name ("," agent_name)*
```

A *step* is a comma-separated list of agent names; agents in the same step execute concurrently and share the same input. The `->` operator separates steps; the right-hand step begins only after every agent in the left-hand step has finished. The parser is intentionally trivial: a single `split("->")` followed by `split(",")` on each segment, which both keeps the implementation auditable and makes the semantics unambiguous.

Three illustrative examples:

- `"a -> b -> c"`: pure sequential pipeline (semantically equivalent to `SequentialWorkflow`).
- `"a, b, c"`: pure concurrent fan-out (semantically equivalent to `ConcurrentWorkflow`), though note that flow validation requires at least one `->`.
- `"a -> b, c -> d"`: diamond. `a` feeds both `b` and `c` in parallel; both then feed `d`.

The same agent name may appear in multiple steps. `"writer -> reviewer -> writer"` is a valid flow expressing a revise-then-rewrite loop, and the implementation preserves outputs from each occurrence separately so a repeated agent's runs are not silently overwritten.

3.1 Validation Semantics

`validate_flow()` runs at the start of every `run()`. It checks two invariants: (1) the flow contains at least one `->` (preventing accidentally passing a single agent name and getting silent single-shot execution), and (2) every agent name referenced in the flow has been registered in the `agents` dict. Failures raise a `ValueError` with the offending name, surfacing typos immediately rather than letting them silently fall through.

3.2 Conversation as the Shared Bus

A subtle but consequential design choice: every agent in an AgentRearrange flow reads its input from a shared `Conversation` object via `conversation.get_str()`, and writes its output back to the same object. This is true for both sequential and concurrent steps. A sequential agent sees everything that came before it; a parallel agent sees the same prefix as its peers and writes alongside them.

The implication is that information propagates monotonically: each agent sees the full accumulated transcript up to the moment its step begins. This makes the system easy to reason about (there is no hidden message routing) and easy to debug (the conversation log is the complete execution trace).

3.3 Semantics

The shared-conversation discipline above admits a compact formal statement: information accumulates one layer at a time, and within a layer every agent sees the same input. This is the single property from which the correctness of every pattern below follows.

Theorem 1 (Monotone Layered Execution)**.** *Let a flow string parse into an ordered sequence of steps $S_1 \to S_2 \to \cdots \to S_n$, where each step S_k is a non-empty set of agents. Let C_k denote the*

contents of the shared conversation immediately before step S_k begins, with C_1 the initial task. Then for every $k \in \{1, \ldots, n\}$:

1. *every agent $a \in S_k$ reads the same input C_k;*
2. *$C_{k+1} = C_k \cup \{\, \mathrm{out}(a) : a \in S_k \,\}$;*
3. *consequently $C_k = C_1 \cup \bigcup_{j<k} \mathrm{out}(S_j)$, so each agent observes the complete output of every earlier step.*

Proof. By construction. The parser produces the ordered list $(S_1, \ldots, S_n)$ via `split("->")`, and `_run` iterates it in order. Within a step, every agent is dispatched (sequentially when $|S_k| = 1$, or via `run_agents_concurrently` when $|S_k| > 1$) from the same snapshot of the conversation, and outputs are appended only after the step completes; the `Conversation` object is append-only within a run. Properties (1) and (2) are therefore immediate, and (3) follows by induction on k. □

The theorem makes the fan-in behavior of the diamond pattern (Figure 4) a corollary rather than a special case: the downstream agent automatically reads both parallel outputs because they were both appended to C_k before it ran.

4 Architecture and Implementation

The class lives in `swarms/structs/agent_rearrange.py` and clocks in at roughly 1,300 lines including extensive docstrings. The core execution loop is short; most of the file is supporting machinery for telemetry, serialization, and the streaming variants.

4.1 Lifecycle

1. **Construction.** `__init__` stores the agent list as a `dict` keyed by `agent_name`, instantiates a `Conversation` with the configured name and time settings, optionally injects a system-level `rules` message, and runs `reliability_check()` which verifies the agent list, flow, `max_loops`, and `output_type` are non-empty and well-formed.
2. **Optional team awareness.** If `team_awareness=True`, the constructor injects a system message describing the full flow structure (step numbering, predecessors, successors), so agents begin execution already knowing where they sit in the pipeline.
3. **Execution.** `run()` delegates to the private `_run()`, wrapping it in telemetry logging and a try/except that routes failures through `_catch_error()` for graceful logging and autosave.
4. **Step iteration.** `_run` splits the flow on `->` into a list of steps, then walks the list. For each step it splits on `,`: a single-name step dispatches to `_run_sequential_workflow`; a multi-name step dispatches to `_run_concurrent_workflow`.
5. **Concurrent dispatch.** The concurrent helper resolves agent names to objects, calls `run_agents_concurrently` (the framework's parallel runner), and writes each output back to the shared conversation in deterministic order.
6. **Output formatting.** After all steps complete (and possibly after multiple loops, if `max_loops > 1`), `history_output_formatter` converts the conversation to the configured `output_type` (`"all"`, `"final"`, `"list"`, or `"dict"`).

4.2 Sequential Awareness for Repeated Agents

A nuance worth highlighting: when the same agent appears in multiple steps (e.g. `"writer -> reviewer -> writer"`), naive position-by-name lookup would always return the first occurrence, leaving the second invocation thinking it sits at step 0. The implementation threads an explicit `task_idx` through `_get_sequential_awareness`, so each invocation gets correctly contextualized awareness (e.g. "Agent ahead: reviewer" on the second `writer` call). The response dict similarly uses indexed keys (`Writer_0`, `Writer_2`) to preserve outputs from each occurrence.

4.3 Diagram: Internal Control Flow

Figure 1 shows the internal control flow of a single `run()` call.

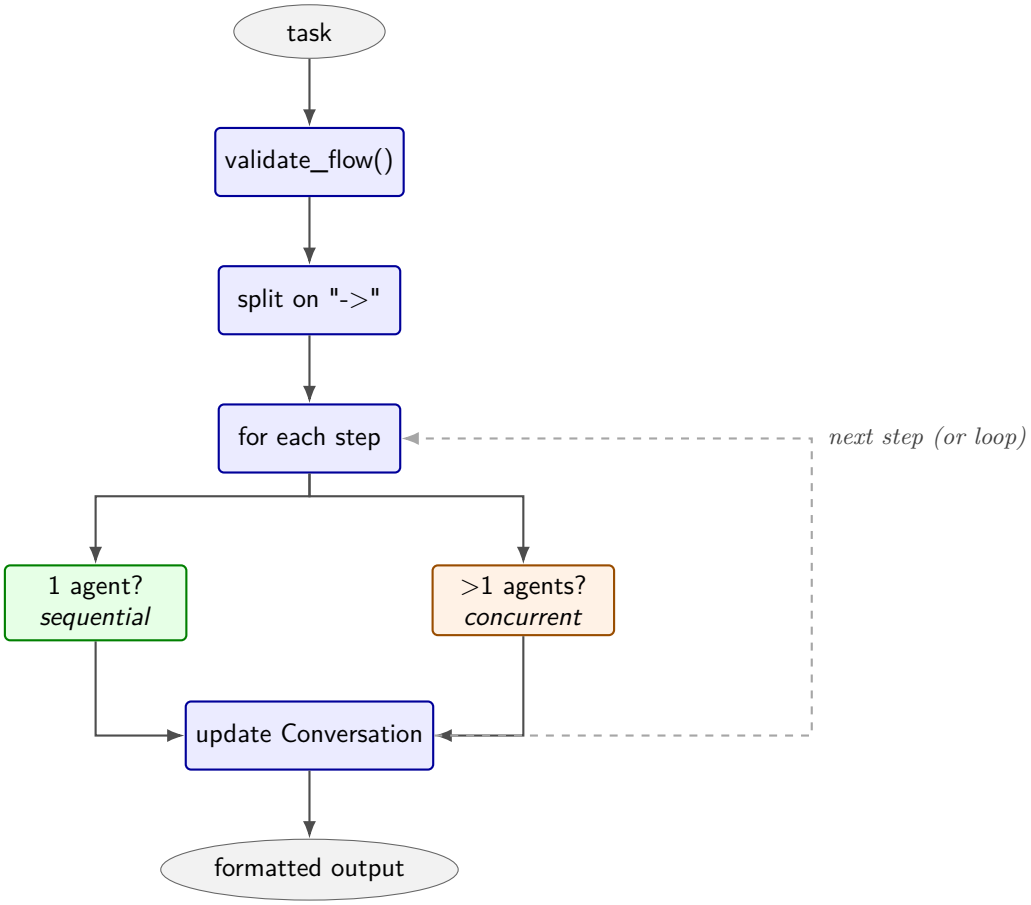

Figure 1: Internal control flow of `AgentRearrange._run()`.

5 Architectural Patterns

The expressive power of AgentRearrange comes from the patterns it makes trivial to write. Four canonical patterns are shown in figures 2–5.

5.1 Linear Pipeline

```
flow = "researcher -> analyst -> writer -> editor"
```

5.2 Fan-out, Fan-in

```
flow = "ingest -> gpt, claude, gemini -> synthesizer"
```

Figure 2: Linear sequential pipeline.

A single source feeds multiple downstream agents in parallel; their outputs converge into a synthesis step. Useful for multi-model ensembling.

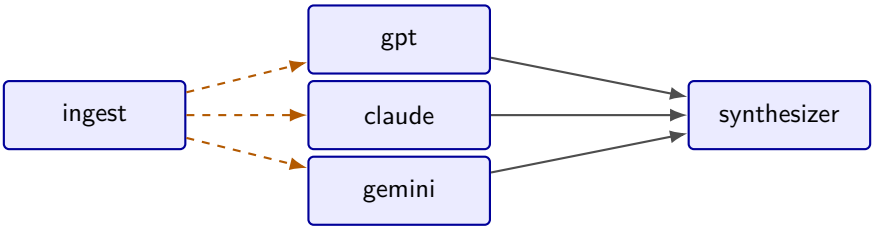

Figure 3: Fan-out / fan-in with multi-model ensembling.

5.3 Diamond

`flow = "planner -> coder, reviewer -> tester"`

The classic diamond shape: one upstream agent, two concurrent middle agents, one downstream aggregator. AgentRearrange handles the join automatically because the shared conversation accumulates both middle outputs before `tester` reads it.

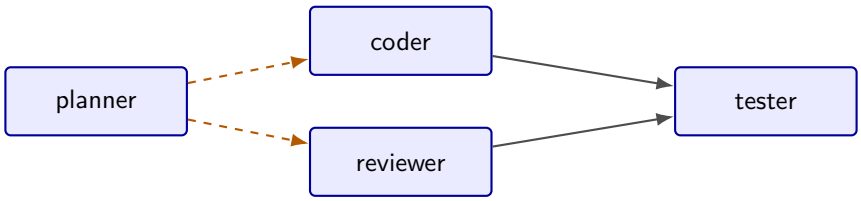

Figure 4: Diamond pattern.

5.4 Human-in-the-Loop Hop

`flow = "drafter -> H -> finisher"` (with `human_in_the_loop=True`)

The `H` marker designates a human step. The `custom_human_in_the_loop` callback receives the upstream output and returns the human's feedback, which is then handed to the downstream agent.

5.5 Compound Pattern

Real workloads combine these. Figure 6 shows a research pipeline where ingestion feeds two parallel analysts, both feed a writer, the writer is reviewed in parallel by an editor and a fact-checker, and a publisher closes the workflow.

```
flow = "ingest -> analyst_a, analyst_b -> writer -> editor, fact_checker ->
publisher"
```

6 Code Examples

6.1 Minimal Sequential

Figure 5: Human-in-the-loop hop.

Figure 6: Compound pattern combining fan-in, fan-out, and final aggregation.

```python
from swarms import Agent, AgentRearrange

researcher = Agent(agent_name="researcher", model_name="gpt-4.1", max_loops=1)
writer     = Agent(agent_name="writer",     model_name="gpt-4.1", max_loops=1)
editor     = Agent(agent_name="editor",     model_name="gpt-4.1", max_loops=1)

system = AgentRearrange(
    agents=[researcher, writer, editor],
    flow="researcher -> writer -> editor",
    max_loops=1,
)
result = system.run("Write an article on the history of transformer architectures.")
```

Listing 1: Minimal sequential pipeline.

6.2 Diamond Workflow

```python
from swarms import Agent, AgentRearrange

planner  = Agent(agent_name="planner",  model_name="gpt-4.1", max_loops=1)
coder    = Agent(agent_name="coder",    model_name="gpt-4.1", max_loops=1)
reviewer = Agent(agent_name="reviewer", model_name="gpt-4.1", max_loops=1)
tester   = Agent(agent_name="tester",   model_name="gpt-4.1", max_loops=1)

pipeline = AgentRearrange(
    agents=[planner, coder, reviewer, tester],
    flow="planner -> coder, reviewer -> tester",
    max_loops=1,
    team_awareness=True,  # tell each agent its neighbors
)
result = pipeline.run("Build a Python function that validates email addresses.")
```

Listing 2: Diamond: one upstream, two concurrent middles, one downstream.

6.3 Multi-Model Ensemble Fan-Out / Fan-In

```python
from swarms import Agent, AgentRearrange

ingest = Agent(agent_name="ingest", model_name="gpt-4.1", max_loops=1)

gpt    = Agent(agent_name="gpt",    model_name="gpt-4.1",
    max_loops=1)
claude = Agent(agent_name="claude", model_name="claude-sonnet-4-6",
    max_loops=1)
gemini = Agent(agent_name="gemini", model_name="gemini/gemini-2.5-pro",
     max_loops=1)

synth  = Agent(
    agent_name="synthesizer",
    model_name="gpt-4.1",
    system_prompt="Combine the three expert responses into one coherent
        answer.",
    max_loops=1,
)

ensemble = AgentRearrange(
    agents=[ingest, gpt, claude, gemini, synth],
    flow="ingest -> gpt, claude, gemini -> synthesizer",
    max_loops=1,
)
print(ensemble("What is the most underrated breakthrough in deep
    learning since 2020?"))
```

Listing 3: Three model providers in parallel, one synthesizer.

6.4 Repeated Agent (Revise Loop)

```python
from swarms import Agent, AgentRearrange

writer   = Agent(agent_name="writer",   model_name="gpt-4.1", max_loops
    =1)
reviewer = Agent(agent_name="reviewer", model_name="gpt-4.1", max_loops
    =1)

revise = AgentRearrange(
    agents=[writer, reviewer],
    flow="writer -> reviewer -> writer",  # same agent, two invocations
    max_loops=1,
    team_awareness=True,
)
result = revise.run("Draft, critique, then revise a short essay on
    Hannah Arendt.")
```

Listing 4: The same writer is invoked twice with a review in between.

6.5 Batched and Concurrent Multi-Task Execution

```python
from swarms import Agent, AgentRearrange

```

```python
a = Agent(agent_name="extract",   model_name="gpt-5.4-mini", max_loops=1)
b = Agent(agent_name="transform", model_name="gpt-5.4-mini", max_loops=1)
c = Agent(agent_name="load",      model_name="gpt-5.4-mini", max_loops=1)

etl = AgentRearrange(
    agents=[a, b, c],
    flow="extract -> transform -> load",
)

tasks = [f"Process document {i}.pdf" for i in range(50)]

# Batched: deep-copies isolate state per task, batch_size at a time.
batch_results = etl.batch_run(tasks=tasks, batch_size=10)

# Concurrent: ThreadPoolExecutor across all tasks at once.
parallel_results = etl.concurrent_run(tasks=tasks, max_workers=8)
```

Listing 5: Running the same flow over many independent tasks.

6.6 Async and Streaming

```python
import asyncio
from swarms import Agent, AgentRearrange

a = Agent(agent_name="a", model_name="gpt-4.1", max_loops=1)
b = Agent(agent_name="b", model_name="gpt-4.1", max_loops=1)
c = Agent(agent_name="c", model_name="gpt-4.1", max_loops=1)

ar = AgentRearrange(agents=[a, b, c], flow="a -> b, c")

async def main():
    # Awaitable result
    result = await ar.run_async("Explain modular arithmetic with examples.")
    print(result)

    # Token-by-token stream; for parallel steps, tokens from b and c interleave.
    async for agent_name, token in ar.arun_stream("Explain it again, briefly."):
        print(f"[{agent_name}] {token}", end="", flush=True)

asyncio.run(main())
```

Listing 6: Async execution and token-level streaming.

7 Advanced Features

7.1 Team Awareness

Setting `team_awareness=True` injects two pieces of context. At construction time, the system message receives a description of the full flow structure: each step numbered, with its predecessors

and successors listed. At execution time, before each sequential agent runs, an additional system message is appended naming the agents directly ahead and behind in the sequence. The intent is to give the agent enough situational awareness to write outputs that the next agent can consume cleanly, without having to encode "you are step 3 of 5" into every prompt by hand.

7.2 Streaming with Fair Interleaving

The streaming variants (`arun_stream`, `run_stream`) yield tokens as they arrive. Sequential steps stream tokens from one agent at a time; parallel steps interleave tokens from concurrent agents via an asyncio queue. A deliberate `await asyncio.sleep(0)` sits between yields in the parallel path; without it the consumer would drain one producer's burst before the scheduler hopped to the next, producing a clumpy `AAAA...BBBB` pattern instead of the fairer `ABABAB` weave users expect from a parallel stream. The `with_events=True` flag swaps the tuple yield for structured event dicts (`agent_start`, `token`, `agent_end`), which is the form most useful for wiring into a server-sent-events handler.

7.3 Batched vs. Concurrent Multi-Task Execution

`batch_run` processes tasks in chunks of `batch_size`, deep-copying the entire orchestrator per task so conversation state is fully isolated. Each chunk runs on a thread pool sized by CPU count. This is the right choice when conversation history must not bleed across tasks; for example, when each task is an independent customer query.

`concurrent_run`, by contrast, fires all tasks into a single `ThreadPoolExecutor` bounded by `max_workers`, with the same orchestrator instance reused. This is faster but assumes the workload tolerates a shared underlying state. For pure I/O-bound flows where every agent calls out to an LLM API, this is usually fine.

7.4 The `rearrange` Convenience Function

For one-shot use, the module exports a `rearrange()` function that constructs an `AgentRearrange`, runs the given task, and returns the result in a single call. It is purely a thin convenience wrapper but reduces a five-line construct to a one-liner for ad-hoc scripts and notebooks.

8 Comparison with Other Swarms Structures

AgentRearrange is the most expressive orchestrator in Swarms but not always the right choice. Table 1 positions it against its siblings.

Structure	Topology	When to prefer it
`SequentialWorkflow`	Linear pipeline	Pure A→B→C; reads marginally cleaner than `AgentRearrange` for that case.
`ConcurrentWorkflow`	Pure fan-out	Same task to N agents, results collected; no downstream join.
`AgentRearrange`	**Arbitrary mix**	**Any time the topology is more than a straight line or a flat fan-out.**
`GraphWorkflow`	Full DAG	When you need per-node callbacks, explicit node IDs, programmatic graph construction, or non-string-expressible dependencies.
`HierarchicalSwarm`	Director + workers	Director must decide *which* workers to invoke per task at runtime, not at flow-definition time.

Table 1: Choosing among Swarms orchestration structures.

The line we draw most often is between `AgentRearrange` and `GraphWorkflow`. The flow DSL covers the 80% case: workflows where the dependency structure can be expressed as a sequence of comma-separated parallel groups. The 20% it does not cover (arbitrary cross-step edges such as a step-3 agent feeding both a step-2 and a step-5 agent, per-node callbacks, or dependency graphs built programmatically from data) is where `GraphWorkflow` earns its weight.

9 Production Considerations

A few practical notes from running AgentRearrange in production.

Agent names must be unique. The agent list is stored as a dict keyed by `agent_name`; duplicates silently overwrite earlier entries. The flow string references agents by name, so a duplicate effectively merges two configurations. Unique, descriptive names are the cheapest reliability investment.

Set explicit `max_loops`. The orchestrator's `max_loops` re-runs the entire flow that many times, with the conversation continuing to accumulate across loops. This is rarely what users want for a single-shot workflow; leave it at the default 1 unless you have a specific iterative refinement pattern in mind.

Watch context length on long flows. Because every agent reads the full conversation, the input to the last agent in a long flow can be very large. For flows of more than 5–6 steps, enable `context_compression=True` on the participating agents so they auto-summarize when nearing the limit.

Validate flows in CI. Because flows are strings, typos slip through. A trivial unit test that instantiates each orchestrator with its production flow and an empty task will surface every "Agent 'X' is not registered" error at test time rather than at 3 a.m.

Use `rules`, not prompt-engineering each agent. The `rules` parameter prepends a system message visible to every agent in the flow. For invariants that apply to the entire workflow ("output JSON only", "never reveal internal tokens"), this is far cleaner than appending the same paragraph to each agent's system prompt.

10 Conclusion and Future Work

AgentRearrange shows that the right abstraction can collapse what would otherwise be a family of orchestrator classes into a single primitive parameterized by a string. The flow DSL is small enough to memorize, expressive enough to capture the workflows that real systems need, and ergonomic enough that engineers iterate on orchestration topology with the same low friction they apply to prompts. The implementation deliberately delegates parallel execution to existing Swarms machinery (`run_agents_concurrently`), keeps the shared `Conversation` as the single source of truth, and exposes sync, async, streaming, batched, and concurrent surfaces without changing the underlying flow.

Several directions remain open. First, the current DSL is linear in steps; expressing fully general DAGs with cross-step edges still requires `GraphWorkflow`. A natural extension is named ports (e.g. `"a -> b -> c, a:tail -> c"`) that let a downstream step pull from any earlier step by name rather than only the immediately preceding one. Second, conditional flows ("if the reviewer flags an issue, loop back to writer; otherwise advance to publisher") currently require leaving the DSL behind for Python control flow; a minimal conditional operator would absorb this back into the string. Third, the team-awareness payload could be cached and templated rather than reconstructed per step, removing a small but real source of token overhead on long flows.

The framework is open-source under the Apache-2.0 license. The source for AgentRearrange lives at `swarms/structs/agent_rearrange.py` in the main Swarms repository. Issues, feature requests, and pull requests are welcome.

Code and Resources

Resource	Location
Source repository	github.com/kyegomez/swarms
AgentRearrange source	`swarms/structs/agent_rearrange.py`
Documentation	docs.swarms.world
PyPI package	pypi.org/project/swarms
Marketplace	swarms.world
Community (Discord)	discord.gg/EamjgSaEQf
Twitter / X	@swarms_corp
License	Apache-2.0
Version	Swarms 12.0.0
Contact	kye@swarms.world

References

[1] Wu, Q., Bansal, G., Zhang, J., Wu, Y., Zhang, S., Zhu, E., Li, B., Jiang, L., Zhang, X., & Wang, C. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv preprint arXiv:2308.08155*, 2023.

[2] Hong, S., Zheng, X., Chen, J., Cheng, Y., Wang, J., Zhang, C., Wang, Z., Yau, S.K.S., Lin, Z., Zhou, L., Ran, C., Xiao, L., Wu, C., & Schmidhuber, J. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. In *International Conference on Learning Representations (ICLR)*, 2024.

[3] Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

[4] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*, 2023.

[5] Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[6] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[7] Wang, J., Wang, J., Athiwaratkun, B., Zhang, C., & Zou, J. Mixture-of-Agents Enhances Large Language Model Capabilities. *arXiv preprint arXiv:2406.04692*, 2024.

[8] Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., & Narasimhan, K. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[9] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., & Zhou, D. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[10] Significant Gravitas. AutoGPT: An Autonomous GPT-4 Experiment. GitHub repository, 2023. https://github.com/Significant-Gravitas/AutoGPT.

[11] LangChain, Inc. LangGraph: Building Stateful, Multi-Actor Applications with LLMs. GitHub repository, 2024. https://github.com/langchain-ai/langgraph.

[12] crewAI Inc. CrewAI: Framework for Orchestrating Role-Playing, Autonomous AI Agents. GitHub repository, 2024. https://github.com/crewAIInc/crewAI.

[13] BerriAI. LiteLLM: Call All LLM APIs Using the OpenAI Format. GitHub repository, 2024. https://github.com/BerriAI/litellm.

[14] Anthropic. Introducing the Model Context Protocol. 2024. https://www.anthropic.com/news/model-context-protocol.

[15] OpenAI. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774*, 2023.

[16] Anthropic. The Claude 3 Model Family: Opus, Sonnet, Haiku. Technical report, Anthropic, 2024.

[17] Apache Software Foundation. Apache Airflow: A Platform to Programmatically Author, Schedule, and Monitor Workflows. 2014–present. https://airflow.apache.org.

[18] Gomez, K. and the Swarms Research Department. Swarms: The Enterprise-Grade Production-Ready Multi-Agent Orchestration Framework. GitHub repository, version 12.0.0, 2024–2026. https://github.com/kyegomez/swarms.